

Laboratory work No. 1
«Roots of equations 8.34»

Submitted by
Farrakhetdinova Azaliya
19/09/2025

Problem statement

Beyond the *Colebrook equation*, other relationships, such as the *Fanning friction factor* f , are available to estimate friction in pipes. The Fanning friction factor is dependent on a number of parameters related to the size of the pipe and the fluid, which can all be represented by another dimensionless quantity, the *Reynolds number* Re . A formula that predicts f given Re is the *von Karman equation*,

$$\frac{1}{\sqrt{f}} = 4 \log_{10}(Re\sqrt{f}) - 0.4$$

Typical values for the Reynolds number for turbulent flow are 10 000 to 500 000 and for the Fanning friction factor are 0.001 to 0.01. Develop a function that uses bisection to solve for f given a user-supplied value of Re between 2 500 and 1 000 000. Design the function so that it ensures that the absolute error in the result is $E_{a,d} < 0.000005$.

Mathematical model

Mathematical model of the problem consists of the non-linear algebraic equation of the form

$$g(f) = 0, \tag{1}$$

where the function f is defined as follows

$$g(f) = \frac{1}{\sqrt{f}} - 4 \log_{10}(\operatorname{Re}\sqrt{f}) + 0.4$$

Numerical method

The bisection method will be used for numerical solution of the equation (1). The algorithm of the bisection method can be summarized as the follows:

Step 1. Choose two values, a and b , such that $f(a)$ and $f(b)$ have opposite signs. This means a root lies between a and b .

Step 2. Calculate the midpoint c :

$$c = \frac{a + b}{2}$$

Step 3. Evaluate $f(c)$: if $f(c) = 0$, then c is the root. If $f(c) < \varepsilon$ (ε is prescribed accuracy), then c is the root. If not, continue.

Step 4. Determine which sub-interval contains the root: if $f(a)$ and $f(c)$ have opposite signs, root is in $[a, c]$; set $b = c$. Otherwise, set $a = c$.

Step 5. Repeat steps 2–4 until the interval is small enough (desired accuracy).

Result of numerical solution

The main window of interface is shown on Fig.1. There are the input fields, the solution, and the graph output for defining the solution area. If an incorrect area is selected, the program will issue a warning and suggest to choose a different area

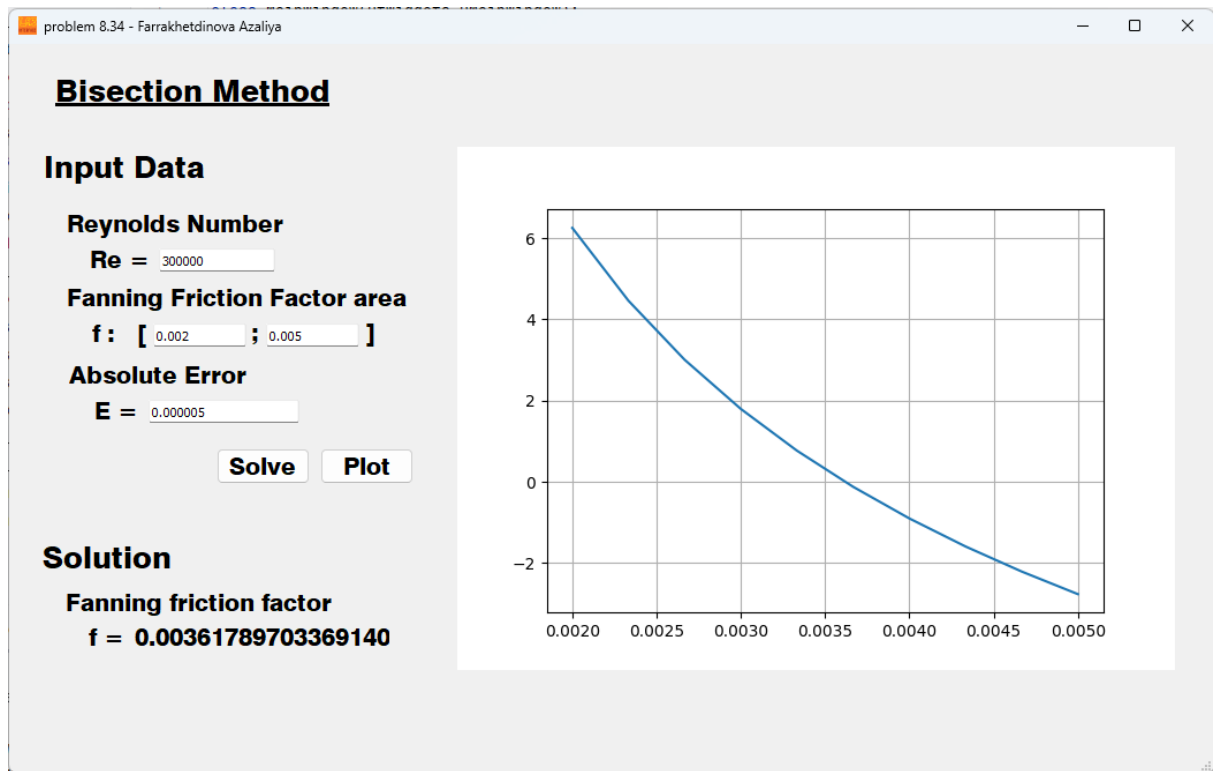


Fig.1 – Main window

Conclusion

For given Reynolds numbers, the program outputs the value corresponding to the equation's solution. This value can be approximated from the graph of the equation's values, where the equation is represented as a function.

Appendix.

A. Main Window Code

```
from PyQt5 import QtWidgets, uic

from solution import Solution

class MainWindow(QtWidgets.QMainWindow):

    def __init__(self):

        super(MainWindow, self).__init__()

        uic.loadUi('interface.ui', self)

        self.solve_button.clicked.connect(self.show_solution)
        self.plot_button.clicked.connect(self.plot_data)

    def plot_data(self):

        self.Re = float(self.re_edit.text())
        self.f_0 = float(self.f_0_edit.text())
        self.f_n = float(self.f_n_edit.text())
        self.Error = float(self.error_edit.text())

        self.solution = Solution(Re=self.Re,
                                f_0=self.f_0,
                                f_n=self.f_n,
                                Error=self.Error)

        self.widget_graph.canvas.axes.clear()
        self.widget_graph.canvas.axes.plot(self.solution.f_arr,
self.solution.var_Func_arr, "-", label='Func(f)')
        self.widget_graph.canvas.axes.grid(True)

        self.widget_graph.canvas.draw()

    def show_solution(self):

        self.Re = float(self.re_edit.text())
        self.f_0 = float(self.f_0_edit.text())
        self.f_n = float(self.f_n_edit.text())
        self.Error = float(self.error_edit.text())

        self.solution = Solution(Re=self.Re,
                                f_0=self.f_0,
                                f_n=self.f_n,
                                Error=self.Error)

        message: str = self.solution.solve()
        print(message)
        self.error_message.setText(message)
        self.f_solved = self.solution.f_n
        self.solution_label.setText(str(self.f_solved))

if __name__ == '__main__':
    app = QtWidgets.QApplication([])
    main = MainWindow()
    main.show()
    app.exec_()
```

B. Bisection Method Solver Class

```
import numpy as np
import math as m

from bisection_method import bisec

class Solution():

    """ Class for solution problem 8-34 with Bisection Method """

    def __init__( self,
                  Re: float,
                  f_0: float,
                  f_n: float,
                  Error: float ):

        self.Re: float = Re
        self.f_0: float = f_0
        self.f_n: float = f_n
        self.Error: float = Error

        self.f_arr: np.ndarray = np.linspace(start=self.f_0, stop=self.f_n,
num=10)
        self.var_Func_arr: np.ndarray = self.Func_arr(self.Re, self.f_arr)

    def Func(self, Re: float, f: float):

        return 1 / m.sqrt(f) - 4 * m.log10(Re * m.sqrt(f)) + 0.4

    def Func_arr(self, Re: float, f: np.ndarray) -> np.ndarray:

        return 1 / np.sqrt(f) - 4 * np.log10(Re * np.sqrt(f)) + 0.4

    def solve(self):

        Func_0: float
        Func_n: float
        Func_middle: float
        cur_error: float = 10000

        if (self.Func(self.Re, self.f_0) * self.Func(self.Re, self.f_n) < 0):

            while cur_error > self.Error:
                Func_0 = self.Func(self.Re, self.f_0)
                Func_n = self.Func(self.Re, self.f_n)
                Func_middle = self.Func(self.Re, (self.f_n + self.f_0) / 2)
                self.f_0, self.f_n = bisec(self.f_0, self.f_n, Func_0,
Func_n, Func_middle)
                cur_error = (abs(self.f_0 - self.f_n) / abs(self.f_n))

            return ''

        else:

            self.f_n = 0
            return 'there is no solution!\n
please, choose different\n
area for f'
```

C. Bisection Method Implementation

```
def bisec(
    a: float,
    b: float,
    f_a: float,
    f_b: float,
    f_middle: float ):

    """ returns new set (a, b) """

    a_new: float = 0
    b_new: float = 0

    if (f_a*f_middle < 0):
        a_new = a
        b_new = (a + b) / 2
    if (f_b*f_middle < 0):
        a_new = (a + b) / 2
        b_new = b

    return a_new, b_new
```

D. Plotting Class (using Matplotlib)

```
import matplotlib
matplotlib.use('Qt5Agg')

from PyQt5 import QtWidgets

from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg
from matplotlib.figure import Figure

class MplCanvas(FigureCanvasQTAgg):

    def __init__(self):
        fig = Figure()
        self.axes = fig.add_subplot(111)
        super().__init__(fig)

class MplWidget(QtWidgets.QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.canvas = MplCanvas()
        self.layout = QtWidgets.QVBoxLayout()
        self.layout.addWidget(self.canvas)
        self.setLayout(self.layout)
```